



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 03

NOMBRE COMPLETO: Alfonso D'Hernán Rodríguez Zuluaga

N° de Cuenta: 321561556

GRUPO DE LABORATORIO: 02

GRUPO DE TEORÍA: 05

SEMESTRE 2026-2

FECHA DE ENTREGA LÍMITE: 08 de marzo del 2026

CALIFICACIÓN: _____

1.- Ejercicio de la pirámide.

```
// Pirámide grande negra
model = glm::mat4(1.0);
//Traslación inicial para posicionar en -Z a los objetos
model = glm::translate(model, glm::vec3(0.0f, 3.0f, -4.0f));
//otras transformaciones para el objeto
model = glm::scale(model, glm::vec3(3.5f, 3.5f, 3.5f));
model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f)); //al presionar la tecla Y se rota sobre el eje y
model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
//La línea de proyección solo se manda una vez a menos que en tiempo de ejecución
//se programe cambio entre proyección ortogonal y perspectiva
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection)); // solo se envía la primera vez que se vaya a dibujar una geometría de un solo color.
glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix())); // solo se actualiza la primera vez, solo se usa una vez más si se cambia de shader
color = glm::vec3(0.0f, 0.0f, 0.0f); // color azul
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
//meshList[0]->RenderMesh(); //dibuja cubo y pirámide triangular
meshList[4]->RenderMeshGeometry(); //dibuja las figuras geométricas cilindro, cono, pirámide base cuadrangular
//sp.render(); //dibuja esfera
```

```
// Pirámide roja 1
model = glm::mat4(1.0);
//Traslación inicial para posicionar en -Z a los objetos
model = glm::translate(model, glm::vec3(0.0f, 3.95f, -3.8f));
//otras transformaciones para el objeto
model = glm::scale(model, glm::vec3(0.9f, 0.9f, 0.9f));
//model = glm::rotate(model, glm::radians(45.0f), glm::vec3(0.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f)); //al presionar la tecla Y se rota sobre el eje y
model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
//La línea de proyección solo se manda una vez a menos que en tiempo de ejecución
//se programe cambio entre proyección ortogonal y perspectiva
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection)); // solo se envía la primera vez que se vaya a dibujar una geometría de un solo color.
color = glm::vec3(1.0f, 0.0f, 0.0f); // color rojo
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
//meshList[0]->RenderMesh(); //dibuja cubo y pirámide triangular
meshList[4]->RenderMeshGeometry(); //dibuja las figuras geométricas cilindro, cono, pirámide base cuadrangular
//sp.render(); //dibuja esfera
```

```
// Pirámide roja 2
model = glm::mat4(1.0);
//Traslación inicial para posicionar en -Z a los objetos
model = glm::translate(model, glm::vec3(-0.5f, 2.95f, -3.2f));
//otras transformaciones para el objeto
model = glm::scale(model, glm::vec3(0.9f, 0.9f, 0.9f));
//model = glm::rotate(model, glm::radians(45.0f), glm::vec3(0.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f)); //al presionar la tecla Y se rota sobre el eje y
model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
//La línea de proyección solo se manda una vez a menos que en tiempo de ejecución
//se programe cambio entre proyección ortogonal y perspectiva
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection)); // solo se envía la primera vez que se vaya a dibujar una geometría de un solo color.
color = glm::vec3(1.0f, 0.0f, 0.0f); // color rojo
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
//meshList[0]->RenderMesh(); //dibuja cubo y pirámide triangular
meshList[4]->RenderMeshGeometry(); //dibuja las figuras geométricas cilindro, cono, pirámide base cuadrangular
//sp.render(); //dibuja esfera
```

```
// Pirámide roja 3
model = glm::mat4(1.0);
//Traslación inicial para posicionar en -Z a los objetos
model = glm::translate(model, glm::vec3(0.0f, 2.0f, -3.2f));
//otras transformaciones para el objeto
model = glm::rotate(model, glm::radians(180.0f), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::rotate(model, glm::radians(52.0f), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::scale(model, glm::vec3(0.9f, 0.9f, 0.9f));
//model = glm::rotate(model, glm::radians(90.0f), glm::vec3(0.0f, 0.0f, 1.5f));
model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f)); //al presionar la tecla Y se rota sobre el eje y
model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
//La línea de proyección solo se manda una vez a menos que en tiempo de ejecución
//se programe cambio entre proyección ortogonal y perspectiva
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection)); // solo se envía la primera vez que se vaya a dibujar una geometría de un solo color.
color = glm::vec3(1.0f, 0.0f, 0.0f); // color rojo
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
//meshList[0]->RenderMesh(); //dibuja cubo y pirámide triangular
meshList[4]->RenderMeshGeometry(); //dibuja las figuras geométricas cilindro, cono, pirámide base cuadrangular
//sp.render(); //dibuja esfera
```

```
// Pirámide roja 4
model = glm::mat4(1.0);
//Traslación inicial para posicionar en -Z a los objetos
model = glm::translate(model, glm::vec3(0.5f, 2.95f, -3.2f));
//otras transformaciones para el objeto
model = glm::scale(model, glm::vec3(0.9f, 0.9f, 0.9f));
//model = glm::rotate(model, glm::radians(45.0f), glm::vec3(0.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f)); //al presionar la tecla Y se rota sobre el eje y
model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
//La línea de proyección solo se manda una vez a menos que en tiempo de ejecución
//se programe cambio entre proyección ortogonal y perspectiva
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection)); // solo se envía la primera vez que se vaya a dibujar una geometría de un solo color.
color = glm::vec3(1.0f, 0.0f, 0.0f); // color rojo
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
//meshList[0]->RenderMesh(); //dibuja cubo y pirámide triangular
meshList[4]->RenderMeshGeometry(); //dibuja las figuras geométricas cilindro, cono, pirámide base cuadrangular
//sp.render(); //dibuja esfera
```

```

// Pirámide roja 5
model = glm::mat4(1.0);
//Traslación inicial para posicionar en -Z a los objetos
model = glm::translate(model, glm::vec3(-1.0f, 2.0f, -2.0f));
//otras transformaciones para el objeto
model = glm::scale(model, glm::vec3(0.9f, 0.9f, 0.9f));
//model = glm::rotate(model, glm::radians(45.0f), glm::vec3(0.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f)); //al presionar la tecla Y se rota sobre el eje y
model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
//La línea de proyección solo se manda una vez a menos que en tiempo de ejecución
//se programe cambio entre proyección ortogonal y perspectiva
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection)); // solo se envía la primera vez que se vaya a dibujar una geometría de un solo color.
color = glm::vec3(1.0f, 0.0f, 0.0f); // color rojo
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
//meshList[0]->RenderMesh(); //dibuja cubo y pirámide triangular
meshList[4]->RenderMeshGeometry(); //dibuja las figuras geométricas cilindro, cono, pirámide base cuadrangular
//sp.render(); //dibuja esfera

// Pirámide roja 6
model = glm::mat4(1.0);
//Traslación inicial para posicionar en -Z a los objetos
model = glm::translate(model, glm::vec3(-0.5f, 1.0f, -2.7f));
//otras transformaciones para el objeto
model = glm::rotate(model, glm::radians(180.0f), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::rotate(model, glm::radians(52.0f), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::scale(model, glm::vec3(0.9f, 0.9f, 0.9f));
//model = glm::rotate(model, glm::radians(90.0f), glm::vec3(0.0f, 0.0f, 1.5f));
model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f)); //al presionar la tecla Y se rota sobre el eje y
model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
//La línea de proyección solo se manda una vez a menos que en tiempo de ejecución
//se programe cambio entre proyección ortogonal y perspectiva
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection)); // solo se envía la primera vez que se vaya a dibujar una geometría de un solo color.
color = glm::vec3(1.0f, 0.0f, 0.0f); // color rojo
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
//meshList[0]->RenderMesh(); //dibuja cubo y pirámide triangular
meshList[4]->RenderMeshGeometry(); //dibuja las figuras geométricas cilindro, cono, pirámide base cuadrangular
//sp.render(); //dibuja esfera

```

```

// Pirámide roja 7
model = glm::mat4(1.0);
//Traslación inicial para posicionar en -Z a los objetos
model = glm::translate(model, glm::vec3(0.0f, 2.0f, -2.0f));
//otras transformaciones para el objeto
model = glm::scale(model, glm::vec3(0.9f, 0.9f, 0.9f));
//model = glm::rotate(model, glm::radians(45.0f), glm::vec3(0.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f)); //al presionar la tecla Y se rota sobre el eje y
model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
//La línea de proyección solo se manda una vez a menos que en tiempo de ejecución
//se programe cambio entre proyección ortogonal y perspectiva
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection)); // solo se envía la primera vez que se vaya a dibujar una geometría de un solo color.
color = glm::vec3(1.0f, 0.0f, 0.0f); // color rojo
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
//meshList[0]->RenderMesh(); //dibuja cubo y pirámide triangular
meshList[4]->RenderMeshGeometry(); //dibuja las figuras geométricas cilindro, cono, pirámide base cuadrangular
//sp.render(); //dibuja esfera

// Pirámide roja 8
model = glm::mat4(1.0);
//Traslación inicial para posicionar en -Z a los objetos
model = glm::translate(model, glm::vec3(0.5f, 1.0f, -2.7f));
//otras transformaciones para el objeto
model = glm::rotate(model, glm::radians(180.0f), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::rotate(model, glm::radians(52.0f), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::scale(model, glm::vec3(0.9f, 0.9f, 0.9f));
//model = glm::rotate(model, glm::radians(90.0f), glm::vec3(0.0f, 0.0f, 1.5f));
model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f)); //al presionar la tecla Y se rota sobre el eje y
model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
//La línea de proyección solo se manda una vez a menos que en tiempo de ejecución
//se programe cambio entre proyección ortogonal y perspectiva
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection)); // solo se envía la primera vez que se vaya a dibujar una geometría de un solo color.
color = glm::vec3(1.0f, 0.0f, 0.0f); // color rojo
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
//meshList[0]->RenderMesh(); //dibuja cubo y pirámide triangular
meshList[4]->RenderMeshGeometry(); //dibuja las figuras geométricas cilindro, cono, pirámide base cuadrangular
//sp.render(); //dibuja esfera

```

```

// Pirámide roja 9
model = glm::mat4(1.0);
//Traslación inicial para posicionar en -Z a los objetos
model = glm::translate(model, glm::vec3(1.0f, 2.0f, -2.0f));
//otras transformaciones para el objeto
model = glm::scale(model, glm::vec3(0.9f, 0.9f, 0.9f));
//model = glm::rotate(model, glm::radians(45.0f), glm::vec3(0.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f)); //al presionar la tecla Y se rota sobre el eje y
model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
//La línea de proyección solo se manda una vez a menos que en tiempo de ejecución
//se programe cambio entre proyección ortogonal y perspectiva
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection)); // solo se envía la primera vez que se vaya a dibujar una geometría de un solo color.
color = glm::vec3(1.0f, 0.0f, 0.0f); // color rojo
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
//meshList[0]->RenderMesh(); //dibuja cubo y pirámide triangular
meshList[4]->RenderMeshGeometry(); //dibuja las figuras geométricas cilindro, cono, pirámide base cuadrangular
//sp.render(); //dibuja esfera

```

```

// Pirámide verde 1
model = glm::mat4(1.0);
//Traslación inicial para posicionar en -Z a los objetos
model = glm::translate(model, glm::vec3(0.2f, 3.95f, -4.0f));
//otras transformaciones para el objeto
model = glm::scale(model, glm::vec3(0.9f, 0.9f, 0.9f));
//model = glm::rotate(model, glm::radians(45.0f), glm::vec3(0.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f)); //al presionar la tecla Y se rota sobre el eje y
model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
//la línea de proyección solo se manda una vez a menos que en tiempo de ejecución
//se programe cambio entre proyección ortogonal y perspectiva
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection)); // solo se envía la primera vez que se vaya a dibujar una geometría de un solo color.
color = glm::vec3(0.0f, 1.0f, 0.0f); // color rojo
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
//meshList[0]->RenderMesh(); //dibuja cubo y pirámide triangular
meshList[4]->RenderMeshGeometry(); //dibuja las figuras geométricas cilindro, cono, pirámide base cuadrangular
//sp.render(); //dibuja esfera

// Pirámide verde 2
model = glm::mat4(1.0);
//Traslación inicial para posicionar en -Z a los objetos
model = glm::translate(model, glm::vec3(0.0f, 2.95f, -3.4f));
//otras transformaciones para el objeto
model = glm::scale(model, glm::vec3(0.9f, 0.9f, 0.9f));
//model = glm::rotate(model, glm::radians(45.0f), glm::vec3(0.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f)); //al presionar la tecla Y se rota sobre el eje y
model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
//la línea de proyección solo se manda una vez a menos que en tiempo de ejecución
//se programe cambio entre proyección ortogonal y perspectiva
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection)); // solo se envía la primera vez que se vaya a dibujar una geometría de un solo color.
color = glm::vec3(0.0f, 1.0f, 0.0f); // color rojo
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
//meshList[0]->RenderMesh(); //dibuja cubo y pirámide triangular
meshList[4]->RenderMeshGeometry(); //dibuja las figuras geométricas cilindro, cono, pirámide base cuadrangular
//sp.render(); //dibuja esfera

// Pirámide verde 3
model = glm::mat4(1.0);
//Traslación inicial para posicionar en -Z a los objetos
model = glm::translate(model, glm::vec3(0.0f, 2.8f, -4.0f));
//otras transformaciones para el objeto
model = glm::rotate(model, glm::radians(180.0f), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(-48.0f), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::scale(model, glm::vec3(0.9f, 0.9f, 0.9f));
//model = glm::rotate(model, glm::radians(90.0f), glm::vec3(0.0f, 0.0f, 1.5f));
model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f)); //al presionar la tecla Y se rota sobre el eje y
model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
//la línea de proyección solo se manda una vez a menos que en tiempo de ejecución
//se programe cambio entre proyección ortogonal y perspectiva
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection)); // solo se envía la primera vez que se vaya a dibujar una geometría de un solo color.
color = glm::vec3(0.0f, 1.0f, 0.0f); // color rojo
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
//meshList[0]->RenderMesh(); //dibuja cubo y pirámide triangular
meshList[4]->RenderMeshGeometry(); //dibuja las figuras geométricas cilindro, cono, pirámide base cuadrangular
//sp.render(); //dibuja esfera

// Pirámide verde 4
model = glm::mat4(1.0);
//Traslación inicial para posicionar en -Z a los objetos
model = glm::translate(model, glm::vec3(0.0f, 2.95f, -4.6f));
//otras transformaciones para el objeto
model = glm::scale(model, glm::vec3(0.9f, 0.9f, 0.9f));
//model = glm::rotate(model, glm::radians(45.0f), glm::vec3(0.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f)); //al presionar la tecla Y se rota sobre el eje y
model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
//la línea de proyección solo se manda una vez a menos que en tiempo de ejecución
//se programe cambio entre proyección ortogonal y perspectiva
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection)); // solo se envía la primera vez que se vaya a dibujar una geometría de un solo color.
color = glm::vec3(0.0f, 1.0f, 0.0f); // color rojo
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
//meshList[0]->RenderMesh(); //dibuja cubo y pirámide triangular
meshList[4]->RenderMeshGeometry(); //dibuja las figuras geométricas cilindro, cono, pirámide base cuadrangular

```



```

// Pirámide verde 5
model = glm::mat4(1.0);
//Traslación inicial para posicionar en -Z a los objetos
model = glm::translate(model, glm::vec3(1.2f, 2.0f, -2.9f));
//otras transformaciones para el objeto
model = glm::scale(model, glm::vec3(0.9f, 0.9f, 0.9f));
//model = glm::rotate(model, glm::radians(45.0f), glm::vec3(0.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f)); //al presionar la tecla Y se rota sobre el eje y
model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
//la línea de proyección solo se manda una vez a menos que en tiempo de ejecución
//se programe cambio entre proyección ortogonal y perspectiva
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection)); // solo se envía la primera vez que se vaya a dibujar una geometría de un solo color.
color = glm::vec3(0.0f, 1.0f, 0.0f); // color rojo
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
//meshList[0]->RenderMesh(); //dibuja cubo y pirámide triangular
meshList[4]->RenderMeshGeometry(); //dibuja las figuras geométricas cilindro, cono, pirámide base cuadrangular
//sp.render(); //dibuja esfera

// Pirámide verde 6
model = glm::mat4(1.0);
//Traslación inicial para posicionar en -Z a los objetos
model = glm::translate(model, glm::vec3(1.3f, 1.0f, -3.45f));
//otras transformaciones para el objeto
model = glm::rotate(model, glm::radians(180.0f), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(-48.0f), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::scale(model, glm::vec3(0.9f, 0.9f, 0.9f));
//model = glm::rotate(model, glm::radians(90.0f), glm::vec3(0.0f, 0.0f, 1.5f));
model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f)); //al presionar la tecla Y se rota sobre el eje y
model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
//la línea de proyección solo se manda una vez a menos que en tiempo de ejecución
//se programe cambio entre proyección ortogonal y perspectiva
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection)); // solo se envía la primera vez que se vaya a dibujar una geometría de un solo color.
color = glm::vec3(0.0f, 1.0f, 0.0f); // color rojo
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
//meshList[0]->RenderMesh(); //dibuja cubo y pirámide triangular
meshList[4]->RenderMeshGeometry(); //dibuja las figuras geométricas cilindro, cono, pirámide base cuadrangular

```

```

// Pirámide verde 7
model = glm::mat4(1.0);
//Traslación inicial para posicionar en -Z a los objetos
model = glm::translate(model, glm::vec3(1.2f, 2.0f, -4.0f));
//otras transformaciones para el objeto
model = glm::scale(model, glm::vec3(0.9f, 0.9f, 0.9f));
//model = glm::rotate(model, glm::radians(45.0f), glm::vec3(0.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f)); //al presionar la tecla Y se rota sobre el eje y
model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
//la línea de proyección solo se manda una vez a menos que en tiempo de ejecución
//se programe cambio entre proyección ortogonal y perspectiva
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection)); // solo se envía la primera vez que se vaya a dibujar una geometría de un solo color.
color = glm::vec3(0.0f, 1.0f, 0.0f); // color rojo
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
//meshList[0]->RenderMesh(); //dibuja cubo y pirámide triangular
meshList[4]->RenderMeshGeometry(); //dibuja las figuras geométricas cilindro, cono, pirámide base cuadrangular
//sp.render(); //dibuja esfera

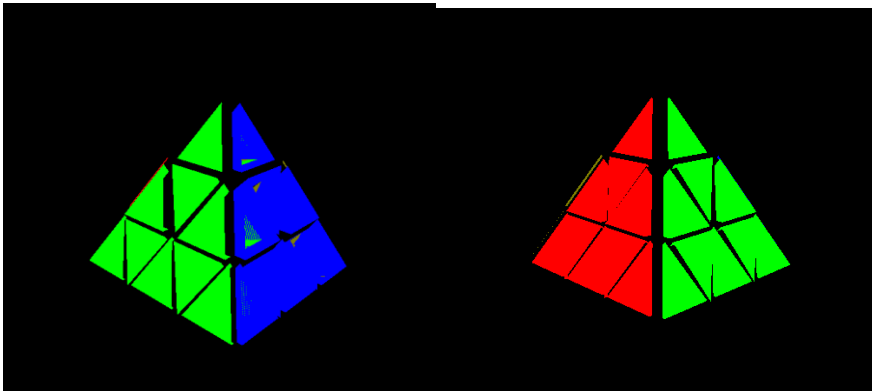
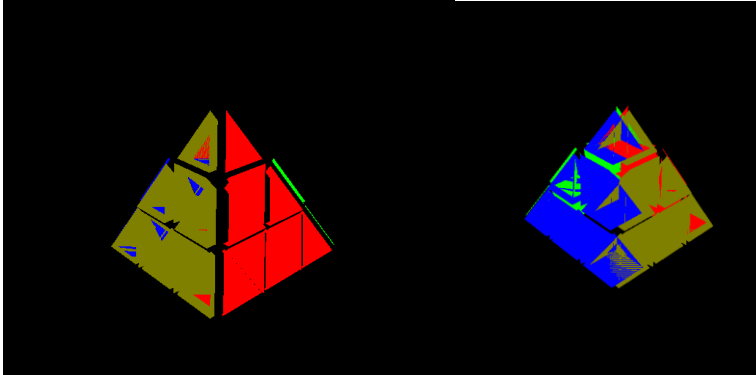
// Pirámide verde 8
model = glm::mat4(1.0);
//Traslación inicial para posicionar en -Z a los objetos
model = glm::translate(model, glm::vec3(1.3f, 1.0f, -4.55f));
//otras transformaciones para el objeto
model = glm::rotate(model, glm::radians(180.0f), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(-48.0f), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::scale(model, glm::vec3(0.9f, 0.9f, 0.9f));
//model = glm::rotate(model, glm::radians(90.0f), glm::vec3(0.0f, 0.0f, 1.5f));
model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f)); //al presionar la tecla Y se rota sobre el eje y
model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
//la línea de proyección solo se manda una vez a menos que en tiempo de ejecución
//se programe cambio entre proyección ortogonal y perspectiva
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection)); // solo se envía la primera vez que se vaya a dibujar una geometría de un solo color.
color = glm::vec3(0.0f, 1.0f, 0.0f); // color rojo
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
//meshList[0]->RenderMesh(); //dibuja cubo y pirámide triangular
meshList[4]->RenderMeshGeometry(); //dibuja las figuras geométricas cilindro, cono, pirámide base cuadrangular

// Pirámide verde 9
model = glm::mat4(1.0);
//Traslación inicial para posicionar en -Z a los objetos
model = glm::translate(model, glm::vec3(1.2f, 2.0f, -5.1f));
//otras transformaciones para el objeto
model = glm::scale(model, glm::vec3(0.9f, 0.9f, 0.9f));
//model = glm::rotate(model, glm::radians(45.0f), glm::vec3(0.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f)); //al presionar la tecla Y se rota sobre el eje y
model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
//la línea de proyección solo se manda una vez a menos que en tiempo de ejecución
//se programe cambio entre proyección ortogonal y perspectiva
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection)); // solo se envía la primera vez que se vaya a dibujar una geometría de un solo color.
color = glm::vec3(0.0f, 1.0f, 0.0f); // color rojo
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
//meshList[0]->RenderMesh(); //dibuja cubo y pirámide triangular
meshList[4]->RenderMeshGeometry(); //dibuja las figuras geométricas cilindro, cono, pirámide base cuadrangular
//sp.render(); //dibuja esfera

```

El código de los lados azul y amarillo es esencialmente el mismo que el de rojo y verde, respectivamente, pero con la diferencia de que se espejean sobre del eje x y del z en cada caso.

Ejecución:



2.- Problemas de los ejercicios.

- Tuve un problema a la hora de tratar de rotar cada uno de los ejes para poder tener las caras en sus ejes respectivos. Pensé que la rotación se podía aplicar de una manera distinta, pues en el efecto de rotación supuse que el hecho de que hubiese 1.0f en cada eje de aplicación significaba que podía aplicar rotaciones parciales (por ejemplo, introducir 90.0f en radianes como argumento pero, para obtener una rotación de 18° , pensé que en la parte de los ejes podía poner 0.2f). Sin embargo, después de probar e investigar un poco, me di cuenta de que los valores de los ejes siempre deben de tener 1.0f en los ejes que se quiere rotar, y que lo que se debe de modificar son los radianes.

- Por alguna razón, las figuras en la imagen de la pirámide están un poco rotas. Traté de volver a extraer la información contenida en el folder enviado por el profe, pero no funcionó. Para prácticas futuras, consideraré simplemente llevar a cabo las prácticas en mi computadora desktop, en vez de mi laptop actual.

3.- Conclusión:

Con relación a los ejercicios dejados como parte del reporte, no encontré que estos fueran muy difíciles, aunque sí encontré que estaba larga la cuestión de implementarlo. Sin embargo, disfruté llevar a cabo el ejercicio una vez que entendí cómo es que debía de implementarlo. No tengo comentarios relativos a la práctica. Considerando que durante la práctica, vimos cómo es que debíamos de implementar rotaciones en los distintos ejes sobre de distintas figura, así como vimos cómo es que se lleva a cabo la construcción de figuras más complejas como conos, esferas y cilindros así como sus ecuaciones matemáticas, considero que la misma cumplió sus objetivos y se completó de manera satisfactoria.

Bibliografía en formato APA:

Román Guadarrama, J. R. (2026). Manual de prácticas: (Práctica No. 03). Universidad Nacional Autónoma de México, Facultad de Ingeniería, División de Ingeniería Eléctrica, Departamento de Computación.